

Codex 的五大核心能力

官方 60 個 Use Cases 深度解讀

理解真實上下文 • 拆解複雜目標 • 呼叫正確工具 • 產出可審查結果 • 在反饋中
持續迭代

國立臺中科技大學 王美智

2026 年 6 月

目 錄

- 一、理解真實上下文
- 二、拆解複雜目標
- 三、呼叫正確工具
- 四、產出可審查的結果
- 五、在反饋中持續迭代

前言

OpenAI Codex 官方更新了 60 個 Use Cases 應用場景，涵蓋從程式碼開發到領域學習的廣泛用途。這份講義整理了其中最核心的五項能力——這五項能力也是 Codex 從「工具」進化為「AI 隊友」的關鍵。

一、理解真實上下文

蝦哥常說：AI 最強的不是會回答，而是「聽得懂」。Codex 的厲害之處在於它能理解你專案當下的真實處境，而不是只會回答泛泛的知識。

官方 60 個 Use Cases 中第一個反覆出現的主題就是「上下文理解」。Codex 在開始工作之前會先讀取專案中的檔案結構，包括：

- AGENTS.md：專案的長期規則檔案，告訴 Codex 這個專案的目標、約束和工作方式。
- 專案目錄結構：Codex 可以遍歷你的專案樹，理解程式碼之間的依賴關係。
- 既有程式碼庫：不只是讀檔案內容，還能理解變數命名風格、架構模式、測試慣例。

這與單純的「問答式 AI」有本質差異——Codex 不是在你的問題之外獨立推理，而是在你的專案之內思考。

► 本章重點

- ► Codex 在工作前會先讀懂你的專案上下文
- ► AGENTS.md 是專案長期記憶的關鍵
- ► 不只是讀檔案，而是理解風格、架構與慣例

？你覺得目前的工作流程中，哪些部分最需要讓 AI 理解上下文？

二、拆解複雜目標

一個模糊的需求是 AI 最大的敵人。Codex 擅長把你說的一句「幫我做個分析工具」拆成一步步可執行的任務。

Codex 60 個 Use Cases 中反覆強調的能力是：將複雜任務分解為可管理的子任務。

舉例來說，當你要求 Codex「分析這份銷售資料並做出儀表板」時，它會自動拆解出：

- Step 1：讀取並理解資料結構
- Step 2：清理與預處理資料
- Step 3：分析關鍵指標（總額、趨勢、異常）
- Step 4：選擇適合的視覺化類型
- Step 5：建立互動式儀表板
- Step 6：加入篩選器與導航功能

這種拆解能力讓複雜專案不再可怕——你不是一個人面對它，Codex 會先幫你把它切分成小塊。

► 本章重點

- ► 模糊需求是 AI 最大的敵人
- ► Codex 能自動將複雜任務拆解為子任務

- ▶ 每個子任務都可獨立執行與驗證

？ 你目前最頭痛的一個專案是什麼？試著讓 Codex 幫你拆解看看。

三、呼叫正確工具

Codex 不只是語言模型——它會根據任務需求自動選擇並呼叫最適合的工具，而不是只用嘴巴回答你。

Codex 的優勢在於它能直接在你的環境中執行命令。官方 60 個 Use Cases 中涵蓋了多種工具呼叫模式：

- 讀寫檔案：建立、修改、刪除專案檔案
- 執行 Shell 命令：安裝套件、執行測試、部署服務
- 呼叫 API：拉取資料、與第三方服務整合
- 執行 linter/test/type checker：自動化品質檢查

真正的 AI 隊友不是給你建議，而是直接動手幫你做。Codex 的「工具呼叫」能力讓它從顧問變成了工程師。

► 本章重點

- ► Codex 能直接在你的環境中執行命令
- ► 支援：讀寫檔案、Shell、API、測試
- ► AI 隊友 = 動手做，不是只給建議

？ 你日常工作中有哪些重複性任務可以讓 Codex 直接代勞？

四、產出可審查的結果

AI 的輸出不可審查，這就是最大的風險。Codex 強調「可審查」——它的產出不是黑盒子，而是你可以檢查、修改、優化的。

官方 60 個 Use Cases 反覆強調的關鍵字是「reviewable」（可審查）。意思是：

- Codex 的產出必須能被人工檢查與驗證
- 每一行程式碼都應有清楚的邏輯與註解
- 變更應可追蹤（diff、版本控制）
- 測試結果應可重現

這與 GPT 的聊天式輸出有本質差異——聊天是消費性的，看完就過去了；Codex 的產出是生產性的，要可以被審查、被修改、被重複使用。

► 本章重點

- ► AI 產出必須可審查、可驗證
- ► 每行程式碼都要有清楚的邏輯
- ► Codex 產出是生產性的，不是消費性的

? 你覺得 AI 的產出要做到什麼程度才算是「可審查」？

五、在反饋中持續迭代

AI 不是一次性的。Codex 真正的價值在於它能在反饋循環中不斷進步——你給它意見，它記住並改進，下次做得更好。

這是 Codex 與一般 AI 助手最顯著的差異：持續迭代。

- 你可以直接修改 Codex 產出的程式碼或檔案
- Codex 會讀取你的修改並從中學習
- 你的反饋會透過 AGENTS.md 和專案結構變成長期記憶
- 每次互動都是「教學相長」——你教 Codex 你的偏好，Codex 幫你做得更好

官方 60 個 Use Cases 的第五個主題就是：在反饋中持續迭代。這不是一個 bug fix 循環，而是一個協作進化機制——你和 Codex 一起變得更好。

► 本章重點

- ► Codex 能在反饋循環中不斷進步
- ► 你的修改就是最好的教學
- ► 人機協作：你教偏好，Codex 優化產出

🔍 你覺得自己與 AI 協作時，最有效的反饋方式是什麼？

參考來源

改寫自 @thinkszyg (爆裂队长 NEXT) 的原文

(<https://x.com/thinkszyg/status/2063805557711188438>)，內容參考 OpenAI

Codex 官方 60 個 Use Cases (<https://developers.openai.com/codex/use-cases>)